



湖南理工学院
Hunan Institute of Science and Technology

课 程： 软件架构体系

班 级： 软件 21-3BF

姓 名： 王卓

学 号： 12214801129

日 期： 2024. 5. 11

建造模式

一. 介绍

建造者模式是一种创建型设计模式，它允许你将一个复杂对象的构建过程与其表示分离，使得相同的构建过程可以创建不同的表示。这个模式的核心思想是将构建复杂对象的过程抽象出来，使得对象的构建步骤可以灵活组合，从而产生不同的对象表示。

在建造者模式中，通常会有以下几个角色：

(1) Director (指挥者)：负责控制构建过程的顺序和流程，即负责调用建造者的方法来构建产品。

(2) Builder (建造者)：定义了构建产品各个部件的抽象接口，具体的建造者类实现这些接口来构建具体的产品。

(3) ConcreteBuilder (具体建造者)：实现了 Builder 接口，负责实际构建产品的各个部件，并返回最终的产品。

(4) Product (产品)：最终构建的复杂对象，通常由多个部件组成。

建造者模式的优点包括：

- 将产品的构建过程与其表示分离，使得可以灵活地组合构建步骤，产生不同的产品。
- 可以隐藏产品的内部结构，使得对客户端而言，产品的构建过程是透明的。
- 可以更好地控制复杂对象的构建过程，使得构建过程更加灵活和可扩展。

二、代码示例

创建一个文档编辑器，其中有不同类型的文档（如简历、报告、新闻稿等），每种文档都有不同的部件（如标题、段落、图片等）。我们可以使用建造者模式来构建不同类型的文档对象。

(1) 产品类-文档

Document 类表示文档产品，具有标题、内容和图片等部件，并有展示详情的方法。

```

1 package builder1;
2
3 //产品类 - 文档
4 class Document {
5     private String title;
6     private String content;
7     private String images;
8
9     public void setTitle(String title) {
10         this.title = title;
11     }
12
13     public void setContent(String content) {
14         this.content = content;
15     }
16
17     public void setImages(String images) {
18         this.images = images;
19     }
20
21     public void show() {
22         System.out.println("Document details - Title: " + title + ", Content: " + content + ", Images: " + images);
23     }
24 }

```

(2) 抽象建造者

DocumentBuilder 接口定义了构建文档的抽象方法，包括构建标题、内容和图片的方法，以及获取最终构建的文档的方法。

```

1 package builder1;
2
3 //抽象建造者类
4 interface DocumentBuilder {
5     void buildTitle();
6     void buildContent();
7     void buildImages();
8     Document getResult();
9 }

```

(3) 指挥者类

DocumentDirector 类负责控制构建文档的顺序和流程，即调用具体建造者类的方法来构建文档。

```

1 package builder1;
2
3 //指挥者类
4 class DocumentDirector {
5     public void constructDocument(DocumentBuilder builder) {
6         builder.buildTitle();
7         builder.buildContent();
8         builder.buildImages();
9     }
10 }

```

(4) 实例化-报告建造者

```

1 package builder1;
2
3 //具体建造者类 - 报告建造者
4 class ReportBuilder implements DocumentBuilder {
5     private Document document = new Document();
6
7     @Override
8     public void buildTitle() {
9         document.setTitle("Report Title");
10    }
11
12    @Override
13    public void buildContent() {
14        document.setContent("Report Content");
15    }
16
17    @Override
18    public void buildImages() {
19        document.setImages("Report Image");
20    }
21
22    @Override
23    public Document getResult() {
24        return document;
25    }
26 }

```

简历建造者:

```

1 package builder1;
2
3 //具体建造者类 - 简历建造者
4 class ResumeBuilder implements DocumentBuilder {
5     private Document document = new Document();
6
7     @Override
8     public void buildTitle() {
9         document.setTitle("Resume Title");
10    }
11
12    @Override
13    public void buildContent() {
14        document.setContent("Resume Content");
15    }
16
17    @Override
18    public void buildImages() {
19        document.setImages("Resume Image");
20    }
21
22    @Override
23    public Document getResult() {
24        return document;
25    }
26 }

```

ResumeBuilder 和 ReportBuilder 类实现了 DocumentBuilder 接口，分别用于构建简历和报告，并设置了不同类型文档的部件配置。

(5) 主程序:

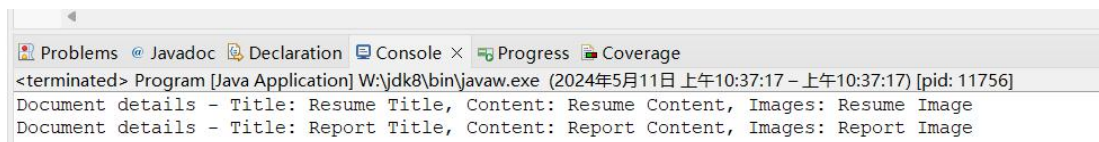
在主程序中，通过创建具体建造者对象（如 ResumeBuilder 和 ReportBuilder）和指挥者对象（DocumentDirector），然后调用指挥者对象的 constructDocument() 方法来构建文档，并展示文档的详情信息。

```

1 package builder1;
2
3 //客户端代码
4 public class Program {
5     public static void main(String[] args) {
6         DocumentDirector director = new DocumentDirector();
7         |
8         DocumentBuilder resumeBuilder = new ResumeBuilder();
9         director.constructDocument(resumeBuilder);
10        Document resume = resumeBuilder.getResult();
11        resume.show();
12
13        DocumentBuilder reportBuilder = new ReportBuilder();
14        director.constructDocument(reportBuilder);
15        Document report = reportBuilder.getResult();
16        report.show();
17    }
18 }

```

三. 运行结果



运行结果显示了两个不同类型文档的详情信息：

- (1) 简历文档的详情信息包括标题为 “Resume Title”，内容为 “Resume Content”，图片为 “Resume Image”。
- (2) 报告文档的详情信息包括标题为 “Report Title”，内容为 “Report Content”，图片为 “Report Image”。
- (3) 在这个例子中，我们使用了建造者模式来构建不同类型的文档对象。具体来说：

ResumeBuilder 和 ReportBuilder 类实现了 DocumentBuilder 接口，分别用于构建简历和报告文档，并设置了不同类型文档的部件配置。

DocumentDirector 类负责控制构建文档的顺序和流程，即调用具体建造者类的方法来构建文档。

在主程序中，我们首先创建了具体建造者对象（ResumeBuilder 和 ReportBuilder）和指挥者对象（DocumentDirector），然后调用指挥者对象的 constructDocument() 方法来构建文档，并展示文档的详情信息。